Sorting Lab

In this lab you implement the sorting algorithms from lecture, then run them side by side on different kinds of input and compare how many operations (comparisons and array writes) each one makes and how long it takes.

Tip: there are many existing implementations of these algorithms including the one you've seen in class. If you just copy-paste those, you'll be done with this lab in no time! You'll also learn nothing. So it's best if you spend this time implementing them yourself.

Before you start, write down which algorithm you expect to be the best on each of the four kinds of input distributions described in Part 3, and why. You don't need to submit it.

How the lab is organized

file	
`sorting/SortingAlgorithm.java`	given: the class every algorithm extends
`sorting/BubbleSort.java`, `SelectionSort.java`, `InsertionSort.java`, `MergeSort.java`	`TODO`
`sorting/QuickSort.java`	`TODO`: quicksort, without a pivot rule
`sorting/QuickSortFirst.java`, `QuickSortMiddle.java`, `QuickSortRandom.java`, `QuickSortMedianOfThree.java`	`TODO`: one pivot rule each
`distributions/`	given: the kinds of input
`Test.java`	given: runs every algorithm on a few small arrays
`Benchmark.java`, `Chart.java`	given: runs the comparison and draws charts
`observations.txt`	`TODO`: your answers to the questions in Part 3

Compile and run from the folder containing `Test.java`:

```
javac Test.java
java Test
javac Benchmark.java
java Benchmark
```

Everything compiles before you've written anything. `Test` shows every algorithm failing, and `Benchmark` skips the algorithms that don't sort yet.

The `SortingAlgorithm` class

Each algorithm implements one method, sorting `a` in place, in ascending order:

```
public void sort(int [] a)
```

It inherits these:

```
protected boolean less(int [] a, int i, int j)   // is a[i] < a[j] ?
protected boolean less(int x, int y)             // is x < y ? for values already out of the array
protected void set(int [] a, int i, int v)     // a[i] = v
protected void swap(int [] a, int i, int j)
```

The benchmark counts **operations**: +1 per comparison, +1 per array write (a swap is two writes), in the inherited field `protected long operations`. `less`, `set` and `swap` count automatically. Writing `a[i] < a[j]` or `a[i] = v` directly still sorts, but goes uncounted: if you prefer to do that, increment `operations` yourself. Writes to a helper array count too.

Part 1: the simple sorts

Implement `BubbleSort`, `SelectionSort`, `InsertionSort` and `MergeSort`, as seen in lecture. Merge sort needs helper methods: add them to the class. After each one, run `Test` and fix it until it prints `OK`.

Part 2: quicksort

`QuickSort` is abstract: it does everything except choose the pivot. Implement `quickSort` and `partition` in `QuickSort.java`, with the partition from lecture: `quickSort` gets the pivot's index from `choosePivot` and passes it to `partition`. Then implement `choosePivot` in each subclass:

- `QuickSortFirst`: the first element;
- `QuickSortMiddle`: the middle element;
- `QuickSortRandom`: a random element (use the `rng` field - you should check the Java docs to see how to draw an integer from it!);
- `QuickSortMedianOfThree`: the median of the first, middle and last elements (compare them with `less`).

`choosePivot` returns an **index** between `left` and `right`, not a value. The stubs return `-1`, so an unimplemented rule fails in `Test` instead of silently doing something else.

Part 3: the comparison

`Benchmark` sorts arrays of N = 50, 100, 200, ..., 25600 elements (each size double the previous), of four kinds, built by the classes in `distributions/`:

- **uniform**: `UniformDistribution`, values drawn uniformly at random from a large range;
- **almost sorted**: `AlmostSortedDistribution`, sorted, then a few elements swapped with close neighbors;
- **almost reverse sorted**: `AlmostReverseSortedDistribution`, the same, then reversed;
- **few unique**: `UniformDistribution` again, with values drawn from only 3 possibilities.

The classes form a chain, each reusing its parent's `generate`: `AlmostSortedDistribution` extends `UniformDistribution` and sorts and perturbs what it generated; `AlmostReverseSortedDistribution` extends `AlmostSortedDistribution` and reverses it.

For each kind of input, `Benchmark` prints two tables, one row per algorithm: the number of operations, and the running time in milliseconds. The last column, `ratio`, is how much the value grew when N doubled from 12800 to 25600.

Try it on small arrays first: `java Benchmark 800` only runs the sizes up to 800, and takes about a second. The full run takes several seconds, most of it the quadratic sorts on the largest arrays.

It also writes charts into `charts/`: operations and time for each kind of input, and operations for each algorithm across all kinds of input. Each chart shows the same data twice. On the left both axes are logarithmic, so a value that grows like N^k is a straight line of slope k. On the right both axes are linear, as you would draw it by hand. The dashed lines marked N, $N \log N$ and N^2 are there to compare against.

Run `Benchmark` and compare with your predictions, using the operation counts. Where your prediction was wrong, explain what actually happens.

Finally, compare the two tables. The times do not always grow the way the operation counts do, especially at small sizes. Why? When they disagree, what else is the time measuring?

Write your answers in `observations.txt`.

Optional: fixing our quicksort for many duplicates

On few unique input, quicksort as we implemented it in class is quadratic, whatever the pivot rule! The problem is `partition`: the right pointer skips every element equal to the pivot, so all of them end up on the pivot's right. Once a subarray contains only equal values, each partition removes just the pivot and leaves everything else on one side.

The fix: let **both** pointers stop on elements equal to the pivot, and swap them. Equal elements are then split between the two sides, and the partitions are balanced again. Two details:

- after a swap, move both pointers "inwards" past the swapped pair: if both elements equal the pivot, neither pointer would move on its own, and the loop would never end;
- stop as soon as the pointers meet or cross, then put the pivot in place as before.

Make the change, check that `Test` still passes, and run `Benchmark` again. What changes on few unique? And on the other inputs? Answer in `observations.txt`.

What to submit

Upload the whole lab folder to Brightspace, with `observations.txt` filled in (leave it where it is).